

Chapter 11

Manipulating Elements and Geometry

The `CurrentLocation` property returns a point object with the `IPoint` interface. In Chapter 9, you wrote code in the mouse down event of the `AddBusStopTool` add-in tool to obtain the x and y coordinates of the point object obtained from the `CurrentLocation` property and displayed the coordinates on the ArcMap status bar.

In this chapter, you will continue to implement the `AddBusStopTool` and `AddBusRouteTool` add-in controls so that they can be used to create graphic elements on the map to represent bus stops and routes respectively. In particular, two types of graphic elements will be introduced, marker elements and line elements. A marker element can represent a point feature such as a bus stop, and a line element can represent a line feature such as a bus route. Once you instantiate a graphic element object, you must set a few key properties such as a geometry, a spatial reference, and a symbol in order to display it on the map. Sections 1 and 2 will use the bus stop and bus route tool examples to demonstrate the process of creating marker and line elements. Section 3 will show you how to create a polyline shape from coordinates in a GPS data file and set it to the line element to display bus routes. Creating and using add-in dockable window will be introduced in Section 3.

1. Creating Marker Elements

ArcMap includes two major categories of elements: frame elements and graphic elements. The former category includes map layout elements such as map frames and map surrounds (map title, legend, scale bar, north arrow, etc.). The graphic element category includes various types of graphics such as marker elements, line elements, polygon elements, and text element. (Figure 1). A graphic element is not a feature stored in a feature class. It can be saved in a map document (.mxd file), and be selected using the select element tool and be deleted using the delete key. This section will go through the process of creating marker elements while implementing the `AddBusStopTool` in the CityGIS project.

1.1 Model diagram analysis and solution development

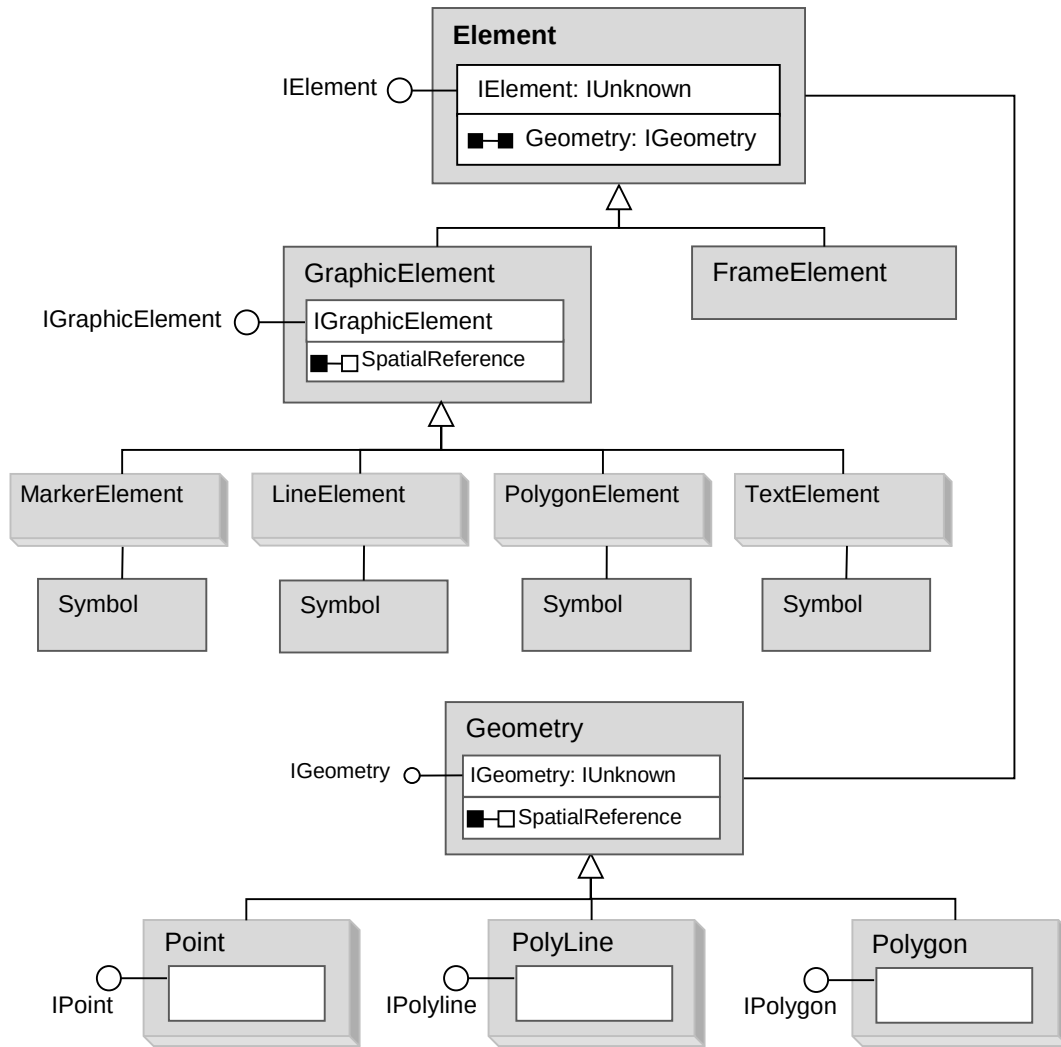


Figure 1 Model diagram for the Element and related classes
(See details on page 2 of “CartoObjectModel.pdf”
and page 1 of “GeometryObjectModel.pdf”)

Figure 1 shows that the Element class and GraphicElement class are abstract classes. The GraphicElement class is inherited by four co-classes: MarkerElement, LineElement, PolygonElement, and TextElement. As their names suggest, different graphic elements are used to display different graphic types. To display a point-type bus stop, we must use the MarkerElement co-class to instantiate a marker element. To display a line-type bus route, we must use the LineElement co-class to instantiate a line element.

Figure 1 also shows that each graphic element co-class is associated with a symbol class. This is because each element co-class has a Symbol property. This property allows us to define the appearance of the element by setting a symbol object to it. Simple marker symbols, simple line symbols and simple fill symbols are introduced in Chapter 10. For convenience of reference, the object model diagram for the symbol class and its co-classes are presented below again (Figure 2).

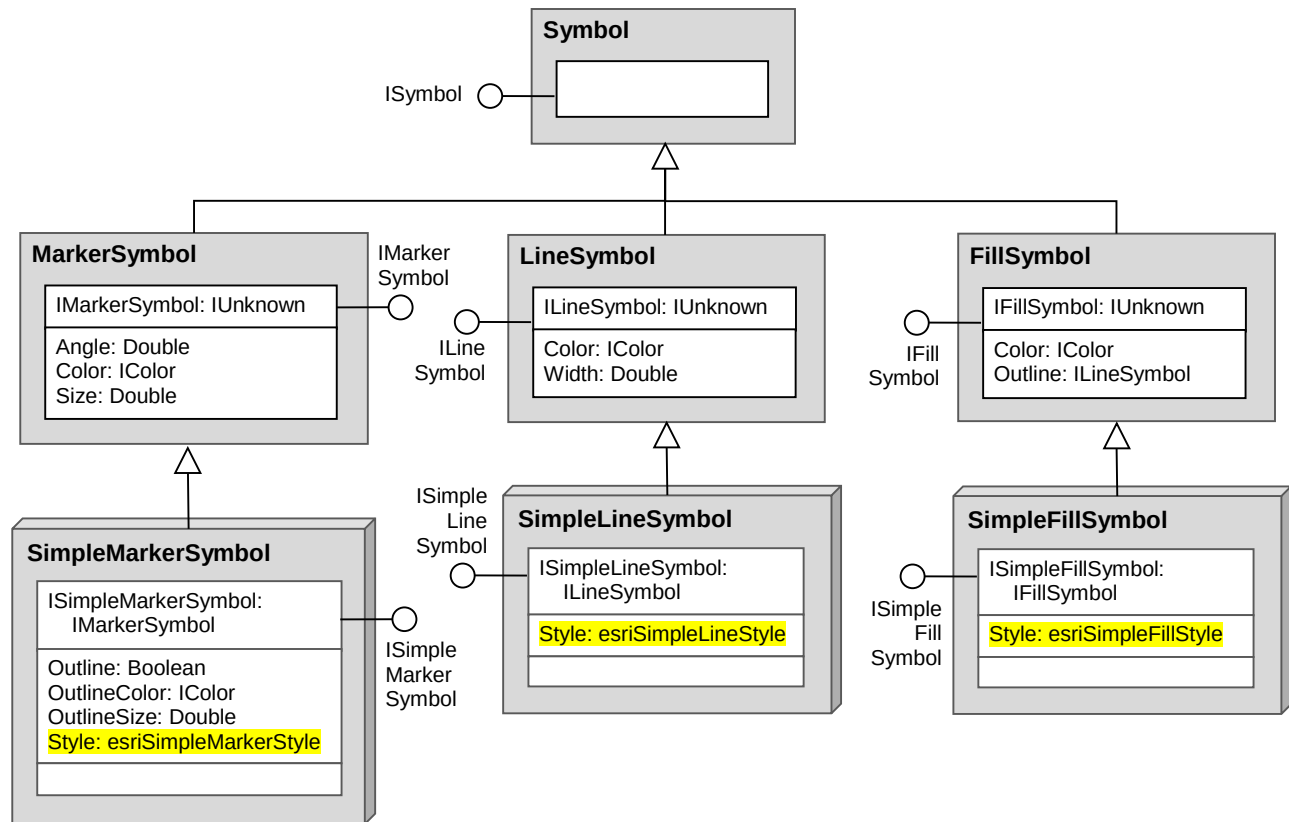


Figure 2 The Symbol abstract class and its child classes
(See details on page 2 in "DisplayObjectModel.pdf")

To use a marker element to represent a bus stop, we must create a simple marker symbol and set it to the element's Symbol property. To use a line symbol element to represent a bus route, we need to create a simple line symbol and set it to the element's Symbol property.

A symbol not only lets us to set properties such as color and a size (or line width) but also lets us to set a style. In Figure 2 each simple symbol co-class has a Style property. This property allows us to select a symbol style from a built-in symbol enumeration. For example, available styles for a simple marker symbol include circle, square, diamond, and cross; available styles for a simple line symbol include solid line, dashed line, and dotted line; and available styles for a simple fill symbol include solid fill, hollow fill, horizontal / vertical / diagonal / cross line fill, etc.

Finally and more importantly, an element itself is not a location or shape, but it has a Geometry property that lets us to set a geometry object (IGeometry interface, see Figure 1). Figure 1 indicates that Geometry is an abstract class. The Geometry class has many co-classes but the diagram only shows three most frequently used ones. To set a geometry object to a bus stop element, we can use the point object obtained from mouse clicking event. The point object derived from the CurrentLocation property has an IPoint interface but we can switch to the IGeometry interface by using QI.

Once an element is created, we can add it into the map to display. This is carried out by calling the AddElement() method on the IGraphicsContainer interface of the Map class (Figure 3). If

we are designing a map layout, we can also add elements to the page layout through the IGraphics Container interface.

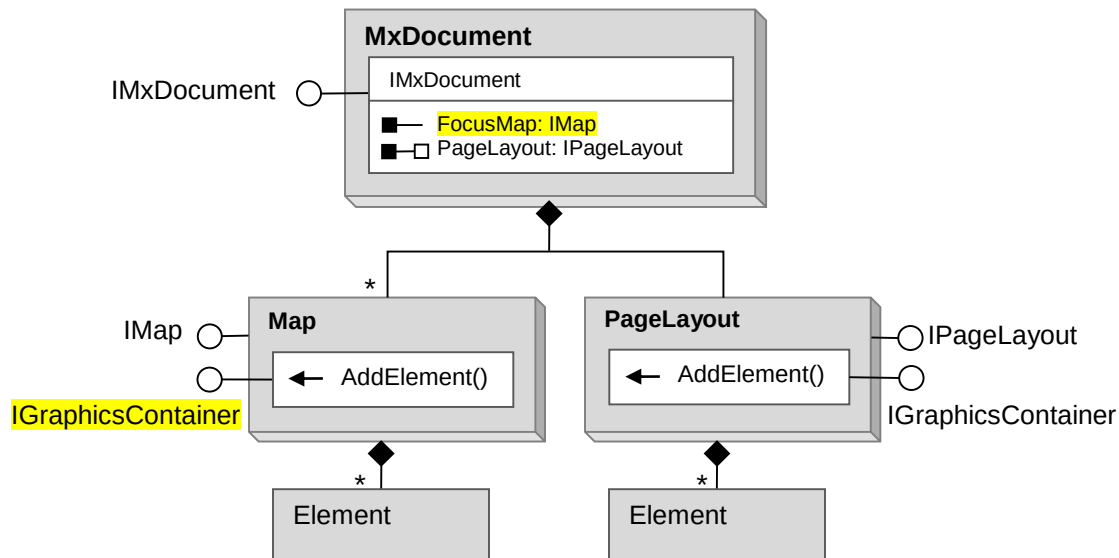


Figure 3 Object model diagram for adding elements into the map

Since the map object returned from the FocusMap property of the MxDocument object points to the IMap interface, a QI is necessary to switch to the IGraphicsContainer interface in order to use the AddElement() method.

To summarize, in order to create a bus stop element using the AddBusStopTool, we need to perform the following tasks in the OnMouseDown event procedure of the tool class:

- Step 1: Get the focus map object (with IMap interface) from the map document (IMxDocument) and QI to the IGraphicsContainer interface.
- Step 2: Get a point object from the CurrentLocation property of the document object (on the IMxDocument interface).
- Step 3: Instantiate a new element object (with IMarkerElement interface) from the MarkerElement co-class.
- Step 4: QI to the IElement interface and assign the point object to the Geometry property on IElement interface.
- Step 5: Instantiate a symbol object from the ISimpleMarkerSymbol co-class, and assign it to the Symbol property on the IMarkerElement interface
- Step 6: Add the marker element object into the map by calling the AddElement() method on the IGraphicsContainer interface
- Step 7: Refresh the map.

1.2 Implementing the bus stop creation tool



```

Protected Overrides Sub OnMouseDown(...)
    ' Step 1:
    ' a. get the MxDocument object (to land at the airport)
    ' b. get the focus map from the map document
    ' c. QI to the IGraphicsContainer interface
    Dim mxDoc As IMxDocument = My.ArcMap.Document
  
```

```

Dim map As IMap = mxDoc.FocusMap
Dim gc As IGraphicsContainer = map ' QI

' Step 2: get the mouse location by calling the CurrentLocation property
Dim p As IPoint = mxDoc.CurrentLocation

' Step 3: instantiate a new element object from the MarkerElement co-class
Dim mkrElement As IMarkerElement = New MarkerElement()

' Step 4: QI to the IElement interface and set the Geometry property
Dim element As IElement = mkrElement ' QI
element.Geometry = p ' sets the point to the Geometry property. QI implied

' Step 5: Create a simple marker symbol object, and
' set it to the Symbol property on the IMarkerElement interface
' In this step, two custom functions are called to create a simple marker symbol
' 1) call function CreateColor() to create two color objects
Dim fcolor As IColor = CreateColor(100, 255, 100) ' a fill color, QI implied
Dim bcolor As IColor = CreateColor(255, 0, 0) ' a border color, QI implied

' 2) call function CreateMarkerSymbol() to create a simple marker symbol
Dim symbol As ISimpleMarkerSymbol = CreateMarkerSymbol(fcolor, bcolor, 4)
mkrElement.Symbol = symbol ' sets the Symbol property of mkrElement, QI implied

' Step 6: add the element object into the map (IGraphicsContainer interface)
gc.AddElement(element, 0)

' Step 7: partially refresh the map
Dim av As IActiveView = mxDoc.ActivatedView ' QI
av.PartialRefresh(esriViewDrawPhase.esriViewGraphics, element, Nothing)

MyBase.OnMouseDown(arg) ' keep this line at the bottom
End Sub

```

The following are a few key points to note about the code above.

- a) QI is widely used. While some QI's are explicitly performed, others are implicit. An explicit QI uses a line of code to switch interface. After QI, the same object is represented by two variables each pointing to a different interfaces. For example,

```
Dim gc As IGraphicsContainer = map
```

is an explicit QI. After this QI, the map object is represented by two variables: variable *map* points to IMap interface and variable *gc* points to the IGraphicsContainer interface. Since statement "Option Strict On" is not declared for the code module, the TryCast() function is not used in all the QI lines. Otherwise, interface casting is necessary, for example,

```
Dim gc As IGraphicsContainer = TryCast(map, IGraphicsContainer)
```

- b) Implicit QI are popularly used as well especially in setting properties. When a property requires a certain interface of an object, but it is assigned with a different interface of the required object, then an implied QI occurs. For example, in

```
element.Geometry = p
```

the Geometry property of the Element class requires an object with IGeometry interface, but it is assigned with object p which points to the IPoint. Strictly, we need to perform a QI to switch to the IGeometry before assigning object p to the Geometry property. But VB does that for us. You can find several other implicit QI's in the above procedure.

- c) Step 5 creates a simple marker symbol (a little square) object. This is accomplished by calling two custom functions. First, function CreateColor() is called twice to create two color objects (fColor and bColor), one for fill color and the other for border color. Second, a function CreateMarkerSymbol() is called to create the necessary simple marker symbol. This function takes in two color objects and a symbol size value as input, and returns a marker symbol object with ISimpleMarkerSymbol interface. The two custom functions are provided below. Since these functions are very straightforward, no additional explanation is necessary.



```
Private Function CreateMarkerSymbol(fillColor As IColor,
                                   borderColor As IColor,
                                   size As Integer) As ISimpleMarkerSymbol
    Dim mkrSymbol As ISimpleMarkerSymbol = New SimpleMarkerSymbol()
    mkrSymbol.Style = esriSimpleMarkerStyle.esriSMSSquare ' Sets a style
    mkrSymbol.Color = fillColor ' Sets a fill color
    mkrSymbol.OutlineColor = borderColor ' Sets a border color
    mkrSymbol.Size = size ' Sets symbol size
    mkrSymbol.OutlineSize = 1 ' Sets border width
    mkrSymbol.Outline = True ' Makes border visible
    Return mkrSymbol
End Function
```

```
Private Function CreateColor(red As Integer,
                             green As Integer,
                             blue As Integer) As IRgbColor
    Dim color As IRgbColor = New RgbColor
    color.Red = red
    color.Green = green
    color.Blue = blue
    Return color
End Function
```

- d) In step 6, the AddElement(element, z-order) method on the IGraphicsContainer interface has two parameters: the first is an element to add, and the second is a z-order which is an integer number that specifies the vertical order of the element to be placed on the map. Using 0 for the z-order makes the last added element to display on top.
- e) In step 7, the PartialRefresh() method is called to refresh the map. This method allows for refreshing a specific area or layer instead of the entire map. When a bus stop element is added, it only redraws a very small area occupied by the element. Therefore, it is much more efficient than refreshing the whole map. The PartialRefresh() method has three parameters: an esriViewDrawPhase, a data object, and an envelope. Parameter esriViewDrawPhase allows the developer to select a value from an enumeration of built-in constants to specify a view to refresh. In the case of refreshing graphic elements, you need to select constant esriViewGraphics. The data object parameter takes a data object to draw. And the envelope parameter allows the user to specify an envelope (a rectangle) to refresh the specified view. Since a point element does not have an envelope, you can pass object "Nothing" to this parameter.

The process of implementing the bus stop tool demonstrates the entire process of writing code based on object model diagram analysis. Please carefully read the code, comments, and the discussions above and make sure you understand every line of code and can relate code to diagrams.

2. Creating Line Elements

This section will demonstrate how to create line elements through the process of implementing the bus route tool.

2.1 Object model diagram analysis and solution development

You can create line elements to represent a bus route by following the same procedure and using the same object model diagrams (Figures 1 and 2). The main difference lies in that you must create a polyline geometry in order to set the geometry property of the line element. With ArcObjects, a polyline object can be derived from a rubber band object (Figure 4).

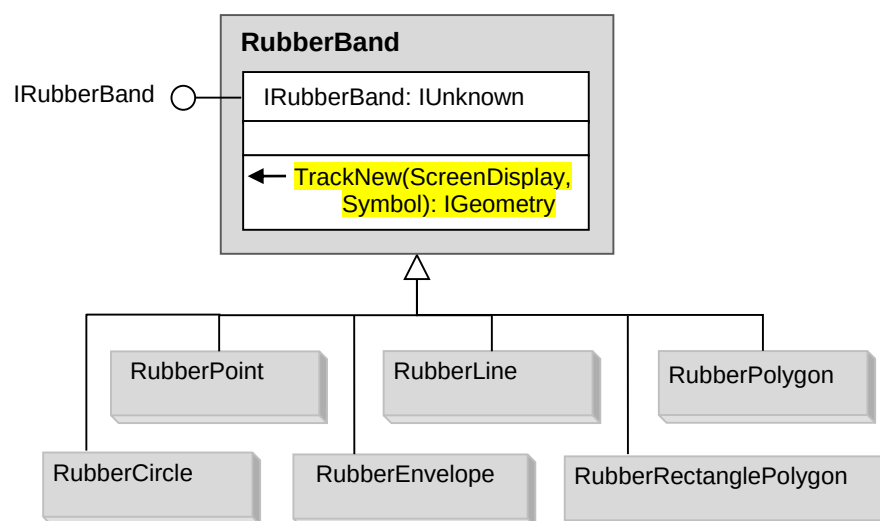


Figure 4 The RubberBand abstract class and its child co-classes
(See details on page 1 of "DisplayObjectModel.pdf")

The IRubberBand interface of the RubberBand abstract class has a method named TrackNew() that returns a geometric object with IGeometry interface (Figure 4). This method requires two arguments as input: a screen display object and a symbol object. A screen display object can be derived from the IActiveView interface of the map object (Figure 5). This object contains functions for drawing graphics and transforming coordinates between screen and map. The symbol object is supposed to symbolize the tracking shapes, but currently the TrackNew() method somehow does not honor it. Therefore we may simply pass "Nothing" for this parameter.

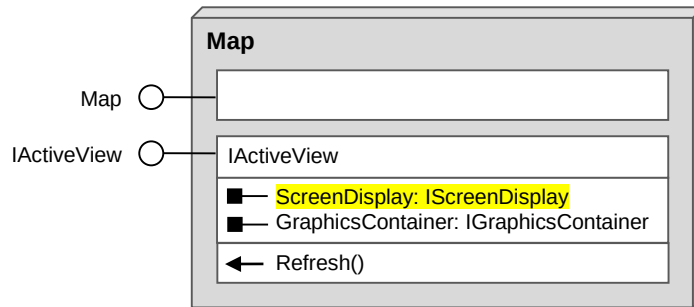


Figure 5 The ScreenDisplay property on the IActiveView interface of the Map class

Finally, we can draw a roadmap to the implementation of the OnMouseDown event procedure of the AddBusRouteTool add-in tool.

- Step 1: Get the screen display object from the ScreenDisplay property on the IActiveView interface of the focus map.
- Step 2: Instantiate a rubber band object from the RubberLine co-class, and call the TrackNew() method to track a line object.
- Step 3: Create a simple line symbol object (for use in step 4) by calling a custom function to be established.
- Step 4: Instantiate a line element object from the LineElement co-class and set the line symbol object created in step 3 to its Symbol property.
- Step 5: QI to the IElement interface of the element and set the Geometry property by assigning the tracked line object to it.
- Step 6: Call the AddElement() method on IGraphicsContainer interface to add the element into the map.
- Step 7: Refresh the map by calling the PartialRefresh() method on IActiveView interface of the map.

2.2 Implementing the bus route tool



```

Protected Overrides Sub OnMouseDown(...)
    ' Step 1: get the ScreenDisplay object from the map (IActiveView interface)
    Dim av As IActiveView = My.ArcMap.Document.ActiveView
    Dim screenDisplay As IScreenDisplay = av.ScreenDisplay

    ' Step 2:
    ' a. instantiate a new rubber band object from the RubberLine co-class
    Dim rubberLine As IRubberBand = New RubberLine()
    ' b. track a shape by calling the TrackNew() method of the rubber band object
    Dim shape As IGeometry = rubberLine.TrackNew(screenDisplay, Nothing)
    If shape Is Nothing Then Return ' exits the sub if a shape is not tracked

    ' Step 3: Create a simple line symbol from a custom function to be created
    Dim color As IColor = CreateColor(255, 0, 0) ' create a color
    Dim lineSymbol As ISimpleLineSymbol = CreateLineSymbol(color, 1.75)

    ' Step 4: instantiate a new line element and set its symbol property
    Dim lineElement As ILineElement = New LineElement
    lineElement.Symbol = lineSymbol ' set the symbol property
  
```



```

' Step 5: QI to the IElement interface and set the Geometry property
Dim element As IElement = lineElement ' QI to the IElement interface
element.Geometry = shape ' sets the geometry property of the element

' Step 6: Add the line element into the map
Dim gc As IGraphicsContainer = av ' QI to the IGraphicsContainer interface
gc.AddElement(element, 0) ' adds the element into the graphics container

' Step 7: refresh the map
av.PartialRefresh(esriViewDrawPhase.esriViewGraphics, shape, shape.Envelope)

MyBase.OnMouseDown(arg) ' keep this line at the bottom
End Sub

```

Two functions, CreateColor() and CreateLineSymbol(), are called in this event procedure. For the CreateColor() function, you can copy it from the AddBusStopTool class and paste it into the current (AddBusRouteTool) class. Function CreateLineSymbol() is provided below. Please add it into the current class. Refer to Figure 2 when you create the function.



```

Private Function CreateLineSymbol(color As IColor,
                                width As Double) As ISimpleLineSymbol
    Dim symbol As ISimpleLineSymbol = New SimpleLineSymbol
    symbol.Color = color
    symbol.Width = width
    Return symbol
End Function

```

In step 2 of the event procedure, variable *shape* is declared as IGeometry interface. It is used to receive the line object returned by the TrackNew() method which produces a geometry object with IGeometry interface. Since variable *shape* points to IGeometry interface, it does not have properties and methods of a polyline object although we know it is a polyline because the rubber band object was instantiated from the RubberLine class. If you want to use any property (e.g. length) or method on the IPolyline interface, you must perform a QI to switch to that interface. To report the track line length, for example, you can add the following two lines of code at the bottom of the OnMouseDown event procedure of the AddBusRouteTool class.

```

Dim polyline As IPolyline = shape ' QI to the IPolyline interface
MsgBox("Bus Route length: " & polyline.Length())

```

Now, you can run the program to test the add bus route tool. Double click on the map to stop tracking.

3. Displaying GPS Data as Line Elements

The AddBusStopTool and AddBusRouteTool could be used to manually create tentative bus stops and routes in transportation planning. In Chapter 5, the BusRouteAnalyst project (Example 4) demonstrates a process to extract coordinates from a GPS data file and computing the length of bus routes. This section will allow you to combine the two capabilities to display bus routes from data parsed from the GPS data file. You will create a new add-in button and an add-in dockable window in the CityGIS project. The add-in button will be used to launch the dockable window in which the main functionality will be implemented



To get started, please create a new add-in button named “DrawGPSRoutesButton” and add it into the CityGIS toolbar. You may use image  (find it on the CD-ROM) for this button. If you need help for these tasks, you can refer to Section 1.2 of Chapter 10.

3.1 Creating an add-in dockable window



- 1) Right click on the project title CityGIS in Solution Explorer and select Add >> New Item
- 2) In the installed templates list, select ArcGIS >> Desktop Add-ins, then enter a name “BusRoutesDockableWindow” in the name box at the bottom and click the Add button
- 3) In the ArcGIS Add-ins wizard dialog, select “Dockable window” in the Add-in component list, and configure the dockable window as Figure 6.

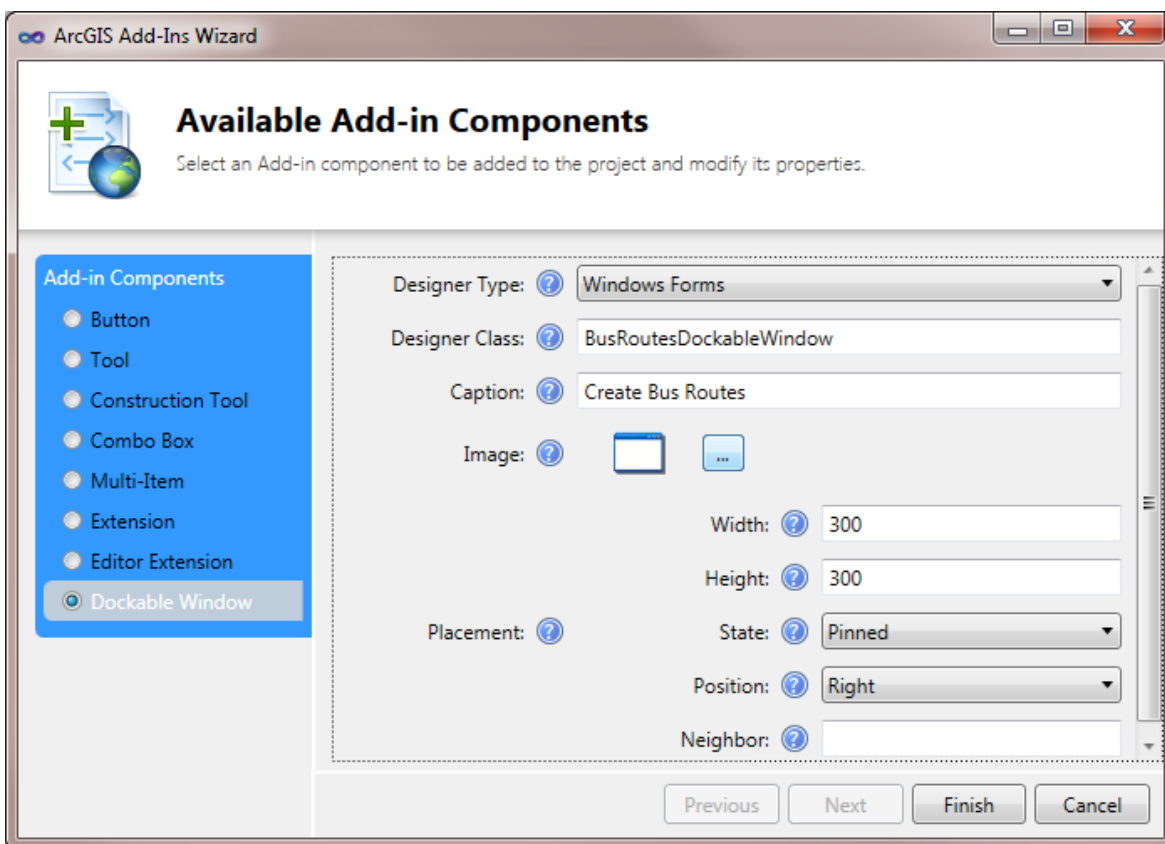
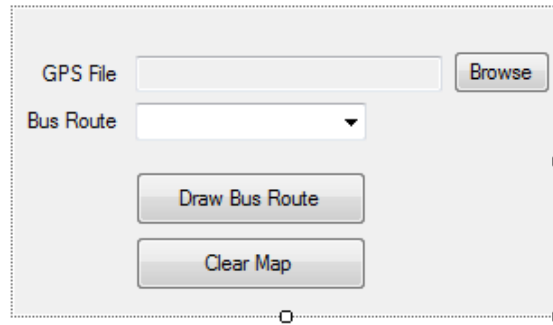


Figure 6 Configuring an add-in dockable window

- 4) Design the GUI of the dockable window using the following settings (Figure 7):



Control	Name	Other Properties
Window	BusRoutesDockableWindow	Text: Draw Bus Routes
Text box	txtFile	ReadOnly: True Anchor: Top, Left, Right
Combo box	cboRoutes	Text: Select a bus route
Button	btnBrowse	Text: Browse Anchor: Top, Right
Button	btnDrawBusRoute	Text: Draw Bus Route
Button	btnClearMap	Text: Clear Map

Figure 7 The draw bus route dockable window user interface

The Anchor property allows a control's specified borders to be anchored when the window is resized. Once you have created the dockable window and the add-in button (DrawGPSRoutesButton), please import namespaces ESRI.ArcGIS.Framework and ESRI.ArcGIS.esriSystem into the add-in button's code module, and then add the following code in the OnClick() event procedure of the add-in button:



```

' instantiate a UID object for the dockable window
Dim WinUID As New ESRI.ArcGIS.esriSystem.UID ' class UID in esriSystem assembly
WinUID.Value = My.ThisAddIn.IDs.BusRoutesDockableWindow ' sets the value property

' get the application object and QI to the IDockableWindowManager interface
Dim app As ESRI.ArcGIS.Framework.IApplication = My.ArcMap.Application
Dim docwinManager As IDockableWindowManager = app ' QI

' get the dockable window from the application object (IDockableWindowManager)
Dim dockwin As IDockableWindow = docwinManager.GetDockableWindow(WinUID)
dockwin.Show(True) ' displays the dockable window

```

Every COM object has a unique identifier (UID) so does every ArcObjects component. In ArcObjects programming, any user-created or built-in toolbar, button, tool, menu items, dockable window, etc. can be accessed via its UID. UIDs of user-created components are managed by a class named "IDs" in the "ThisAddIn" module under the "My" namespace. Therefore, it is referenced by My.ThisAddIn.IDs in which namespace "My", module "ThisAddIn", and class "IDs" are defined in your project by some hidden code generated by the ArcGIS add-in wizard. The second line of code above gets the UID value (a string) of the dockable window from the IDs class via My.ThisAddIn.IDs.BusRoutesDockableWindow, and then sets it to the Value property of a new UID object (WinUID).

A user-created dockable window is managed by the ArcGIS Application (ArcMap in this case). The IDockableWindowManager interface of the Application object has a method named GetDockableWindow() which takes a UID object as input and returns a dockable window object. Once you get a dockable window object, you can call its Show() method to open it by passing a Boolean value (True) to the method as an argument. To hide a dockable window, you can pass value False to the Show() method.

The UID class exists in the esriSystem assembly (ESRI.ArcGIS.esriSystem, in the System model diagram), and the two interfaces of the application object (IApplication and IDockableWindow) are included in the Framework (ESRI.ArcGIS.Framework) assembly. Both assemblies are already loaded into the project as references by the add-in wizard. You may import their namespaces into this module for coding convenience.



UIDs of ArcMap built-in commands (including menu items, buttons, tools, etc.) can be found at the ArcObjects Help for .NET website*. To obtain a built-in command, you need to find out its UID (CLSID or ProgID) string from the website and create a UID object from it. For example, the CLSID string of the “Zoom In” tool is “{AD1891E4-7C79-11D0-8D7C-0080C7A4557D}” and its ProgID string is “esriArcMapUI.ZoomInTool”. So you can create a UID object by using

```
Dim zoomUID As New ESRI.ArcGIS.esriSystem.UID
zoomUID.Value = "{AD1891E4-7C79-11D0-8D7C-0080C7A4557D}" ' , or
zoomUID.Value = "esriArcMapUI.ZoomInTool"
```

Once you create a UID object for a desired command (tool), you can use the following code to retrieve the command (tool) object. For example,

```
Dim cmdBars As ICommandBars = My.ArcMap.Application.Document.CommandBars
Dim zoomTool As ICommandItem = cmdBars.Find(zoomUID)
```

To execute the command (tool), you can call the Execute() method of the command, e.g.

```
zoomTool.Execute()
```

* <http://resources.arcgis.com/en/help/arcobjects-net/conceptualhelp/index.html#/00010000029s000000>

Run the program to test if you can open the dockable window by clicking the draw GPS routes add-in button on the CityGIS toolbar.

3.2 Creating polyline objects from coordinates

So far, you have learned all skills required for parsing latitude/longitude coordinates from a text file and to display a line element in the map with a geometry tracked from screen clicking. To create a line elements from data in a GPS data file to represent a bus route, the only piece of skill yet to learn is to create a polyline object from a list of coordinates parsed from the file.

Figure 8 shows that we can instantiate a new polyline object with IPolyline interface from the Polyline co-class. Once a new polyline object is created, it is an empty object containing no vertices (points). To add points into the polyline object, we can use the AddPoint() method on the IPointCollection interface. Since the Polyline class inherits the PolyCurve abstract class which provides the IPointCollection interface, we can perform a QI to get the IPointCollection

interface in order to use the AddPoint() method. The AddPoint() methods has a parameter declared as IPoint type, therefore a point object must be passed to it when calling this method.

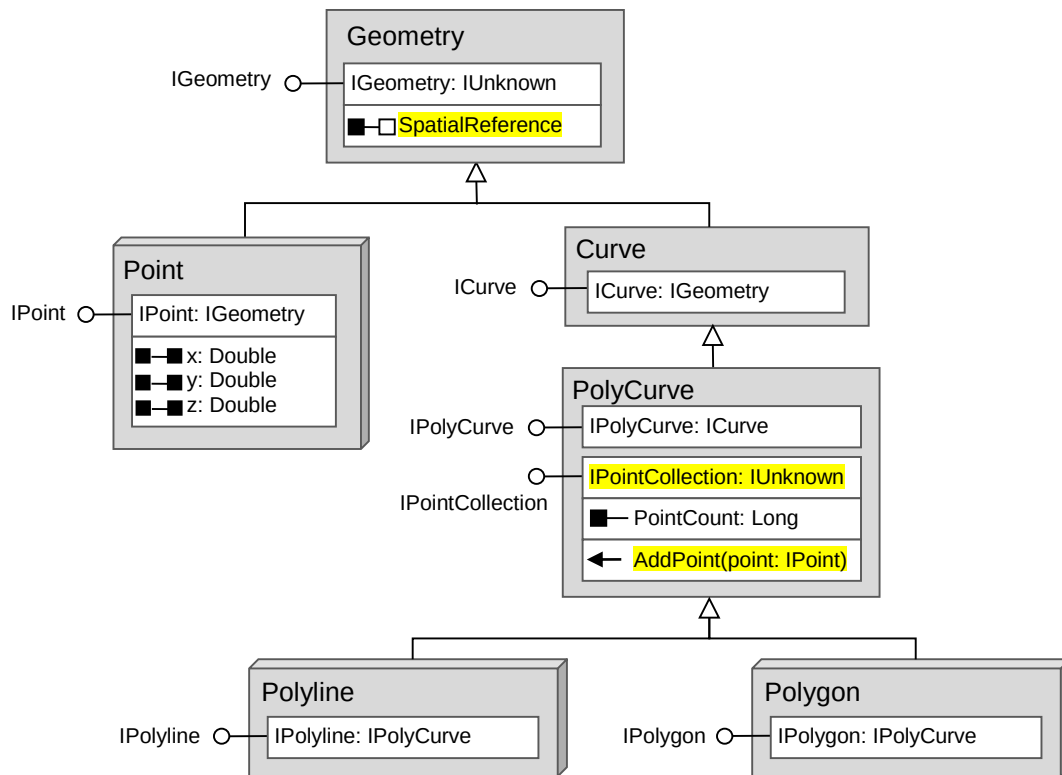


Figure 8 Abridged object model diagram of geometry classes
(See details on page 1 of GeometryObjectModel.pdf)

Suppose we are provided with a series of coordinates for a bus route in two arrays `x()` and `y()`, we can create a function `CreatePolyline()` that makes a polyline object from the arrays (provided below). Since GPS coordinates are recorded in geographic coordinate system (WGS84), when we create a new polyline object from coordinates, we need to specify a geographic coordinate system spatial reference to the polyline so that ArcMap knows how to draw it in a projected map (such as SPCS). To satisfy these needs, a function, `MakeSpatialReferenceGCS()` is also provided below. This function returns a geographic coordinate system (GCS) spatial reference object.

These two functions use several geometry and spatial reference classes (`Point`, `Polyline`, `PolyCurve`, `GeographicCoordinateSystem`, `SpatialReferenceEnvironment`) and interfaces (`IPoint`, `IPolyline`, `IPointCollection`, `ISpatialReference`, `IGeographicCoordinateSystem`) which are in the `Geometry` namespace, so please import the `Geometry` namespace into the code module for coding convenience. Besides, because you will use other `ArcObjects` components including `MxDocument`, `Map`, `Element`, and `Symbol` etc., you will need to import the `Carto` and `Display` namespaces as well. In addition, since you will need to use the `OpenFileDialog` class to create a dialog for the user to browse a GPS data file, and this object exists in namespace `System.Windows.Form`, you will also need to import this namespace. Overall, you need to declare the follow import statements in the header of the `BusRoutesDockableWindow` class module:



```
Imports System.Windows.Forms ' needed by the OpenFileDialog class
Imports ESRI.ArcGIS.Carto    ' needed by MxDocument, Map, ActiveView classes
Imports ESRI.ArcGIS.Geometry ' needed by Geometry classes
Imports ESRI.ArcGIS.Display  ' needed by Symbol and Color classes
```

After adding the above namespace importing statements, you can enter the CreatePolyline() function and MakeSpatialReferenceGCS() function into the BusRoutesDockableWindow class.



```
Private Function CreatePolyline(x As Double(), y As Double()) As IPolyline
    ' 1) instantiate a new polyline object from the Polyline co-class
    Dim polyline As IPolyline = New Polyline

    ' 2) create a Geographic Coordinate System spatial reference object
    Dim srGCS As ISpatialReference = MakeSpatialReferenceGCS _
        (esriSRGeoCSType.esriSRGeoCS_NAD1983)
    polyline.SpatialReference = srGCS ' sets the spatial reference to the polyline

    ' 3) QI to the IPointCollection interface
    Dim pointcoll As IPointCollection = polyline

    ' 4) declare a variable for IPoint interface for use in the loop below
    Dim p As IPoint = Nothing

    ' 5) use a loop to add points to the polyline
    For i As Integer = 0 To x.Length - 1
        p = New Point ' creates a new point object
        p.X = x(i)     ' sets the x property of the point
        p.Y = y(i)     ' sets the y property of the point
        pointcoll.AddPoint(p) ' adds the point into the polyline
    Next
    Return polyline ' returns the polyline object to the client
End Function

Public Function MakeSpatialReferenceGCS(coordinateSystem As esriSRGeoCSType) _
    As ISpatialReference
    ' instantiate a spatial reference factory object
    Dim srFactory As ISpatialReferenceFactory = New SpatialReferenceEnvironment

    ' use the spatial reference factory to create a GCS object
    Dim gcs As IGeographicCoordinateSystem = _
        srFactory.CreateGeographicCoordinateSystem(coordinateSystem)

    ' QI to the ISpatialReference interface
    Dim sr As ISpatialReference = TryCast(gcs, ISpatialReference)

    Return sr
End Function
```

Once you have created the two functions, please copy the CreateLineSymbol() function and CreateColor() function from the AddBusRouteTool class into the BusRoutesDockableWindow class. Then in the Load event procedure of the dockable window, use a loop to add five route numbers (1 to 5) into the route combo box (cboRoutes).

```

For i As Integer = 1 To 5
    cboRoutes.Items.Add(i)
Next

```

Make sure there are no syntax errors and then run the program to test the draw GPS routes button in ArcMap. At this point, nothing will happen when you click the “Draw Bus Route” button in the dockable window because you have not written code for that button. If the program does not crash, it is a good sign and you can continue to implement the button.

3.3 Retrieving coordinates from the GPS data file

Function CreatePolyline() requires coordinates of a bus route as two arrays in order to produce a polyline object. Before implementing the “Draw Bus Route” button, therefore, you need to create another sub procedure that retrieves coordinates of a selected route from the GPS data file and outputs two coordinate arrays.



```

Private Sub ReadRouteCoordinates(ByRef outlats As Double(),
                                ByRef outlons As Double())
    ' create a stream reader object to open the GPS data file
    Dim sr As New IO.StreamReader(txtFile.Text)

    ' read the header line and does nothing on it to consume the file header
    sr.ReadLine()

    ' declare a few variables for use in the loop below
    Dim strLine As String
    Dim data() As String
    Dim counter As Integer = 0

    Do While Not sr.EndOfStream
        ' read a line from the stream reader object
        strLine = sr.ReadLine()
        ' split the line by commas and stores results in the row array
        data = strLine.Split(",")
        If data(1) = cboRoutes.Text Then
            ' data(1) is a route number
            ReDim Preserve outlats(counter) ' expands the latitudes array
            ReDim Preserve outlons(counter) ' expands the longitudes array
            outlats(counter) = Cdbl(data(3)) ' data(3) is a latitude value
            outlons(counter) = Cdbl(data(4)) ' data(4) is a longitude value
            counter += 1 ' increments the counter variable
        End If
    Loop
    sr.Close() ' closes the stream reader
    sr.Dispose() ' disposes the stream reader object
End Sub

```

This procedure declares two array-type parameters. In VB.NET, when you declare an array parameter in a procedure, the pair of parentheses can be placed next to the variable name or next to the data type. Moreover, the two parameters are declared as by-reference parameters with keyword ByRef (ByRef outlats As Double(), ByRef outlons As Double()). This means that once element values in these parameters are updated by the procedure, the updates are made to the arguments in the client procedure. Therefore, using ByRef parameters allows a procedure to send multiple values back to its client. Another thing to note is that since

the two parameters are arrays, even if they are declared with keyword ByVal, they can still behave as by reference parameters. Please refer to Section 5 of Chapter 5.

This function opens the GPS data file using a stream reader object and reads the file line-by-line. After reading a line, it parses data delimited by commas and stores results in an array variable *data*. Element data(1) (the second column of the file) contains a bus route number, data(3) contains a latitude value, and data(4) contains a longitude value. Inside the repetition, if data(1), i.e. the bus route number, matches the user selected bus route in the combo box, the latitude and longitude coordinates are stored in the two coordinate arrays. When the repetition starts, the outlats and outlons arrays are not specified with an upper bound so they cannot hold any value. Once a point on the selected bus route is encountered inside the loop, a new upper bounds (counter) is specified to these arrays using keywords ReDim Preserve, which expands these two arrays. After storing a pair of coordinates, the counter variable is incremented by 1 to get ready for next round of repetition.

Another thing to do before implementing the draw bus route button's Click event procedure is to implement the file browse button. Since you have done this a few times, it should be simple task now. Please see the code below:



```
Private Sub btnBrowse_Click(...) Handles btnBrowse.Click
    ' creat an open file dialog object
    Dim dlgFile As New Windows.Forms.OpenFileDialog
    dlgFile.Title = "Select a GPS Data File" ' set a title
    dlgFile.Filter = "CSV File *.csv|*.csv|" & _
        "Text File *.txt|*.txt" ' sets a filter
    dlgFile.ShowDialog() ' show the dialog

    ' get the file name and send it to the text box
    txtFile.Text = dlgFile.FileName

    ' dispose the dialog object
    dlgFile.Dispose()
End Sub
```

3.4 Final assembly

After getting every necessary part ready, you can perform the final assembly to implement the draw bus route function. The following procedure is similar to what you perform while creating bus routes using the AddBusRouteTool, but with some modifications.

- Step 1: Validate user input, including file name and bus route number.
- Step 2: Call sub procedure ReadRouteCoordinates() to get coordinates of a selected bus route.
- Step 3: Call the CreatePolyline() function to create a polyline object from the coordinates.
- Step 4: Create a line element for the bus route, and then
 - 1) Set the polyline object to the Geometry property on the IElement interface
 - 2) Set a spatial reference, which can be obtained from the polyline object, to the SpatialReference interface on the IGraphicElement interface, and
 - 3) Create a simple line symbol object and set it to the Symbol property on the ILineElement interface
- Step 5: Add the element into the map
- Step 6: Refresh the map.

Use the following code to implement the Click event procedure of the draw bus route button.



```
Private Sub btnDrawBusRoute_Click(...) Handles btnDrawBusRoute.Click
    ' Step 1: validate user input
    If txtFile.Text = "" Then Return
    If Not IO.File.Exists(txtFile.Text) Then
        MessageBox.Show("The file does not exist.", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error)
    End If
    Return
    If Not IsNumeric(cboRoutes.Text) Then Return

    ' Step 2: get coordinates of the selected bus route by calling the
    ' ReadRouteCoordinates() sub procedure and passing two empty arrays to it.
    Dim lats() As Double, lons() As Double ' declares two empty arrays
    ReadRouteCoordinates(lats, lons) ' calls sub ReadRouteCoordinates()
    ' after executing the sub, arrays lats and lons will be filled with values

    ' Step 3: call the CreatePolyline() function to create a polyline object
    ' using coordinates stored in array variables lats and lons.
    Dim busLine As IPolyline = CreatePolyline(lons, lats)

    ' Step 4: create a line element for the bus route
    Dim busElement As ILineElement = New LineElement
    ' 4.1 assign the polyline object to Geometry property of the element
    Dim element As IElement = busElement ' QI to the IElement interface
    element.Geometry = busLine ' sets the geometry property

    ' 4.2 set the spatial reference of the busLine object to the element
    Dim gElement As IGraphicElement = busElement ' QI to IGraphicElement
    gElement.SpatialReference = busLine.SpatialReference ' sets property

    ' 4.3 set the Symbol property of the element
    Dim color As IRgbColor = CreateColor(255, 0, 0) ' creates a red color object
    ' create a line symbol object
    Dim lineSymbol As ISimpleLineSymbol = CreateLineSymbol(color, 2)
    busElement.Symbol = lineSymbol ' sets property

    ' Step 5: get the IActiveView and IGraphicsContainer interfaces of the map
    ' and add the bus element into the map
    Dim av As IActiveView = My.ArcMap.Document.ActiveView
    Dim gc As IGraphicsContainer = av ' QI to IGraphicsContainer
    gc.AddElement(busElement, 0) ' adds element into the map

    ' Step 6: refresh the map by calling the PartialRefresh method (on IActiveView)
    av.PartialRefresh(esriViewDrawPhase.esriViewGraphics, busLine, Nothing)
End Sub
```

There are two key points to pay additional attention in the above code. First, in step 2, sub procedure `ReadRouteCoordinates()` is called to obtain coordinates of a selected bus route. In order to call this sub procedure, two empty Double type array variables *lats* and *lons* are declared and passed to the procedure. Since the `ReadRouteCoordinates()` procedure has two ByRef array parameters (**ByRef** outlats `As Double()`, **ByRef** outlons `As Double()`), after executing this procedure, the array variables *lats* and *lons* will be filled with values.

Second, while creating the element object in step 4, we set three properties to it, each on a different interface: a polyline object is set to the Geometry property on IElement interface (represented by variable *element*), a spatial reference object is set to the SpatialReference property on the IGraphicElement interface (represented by variable *gElement*), and a line symbol object is set to the Symbol property on the ILineElement interface (represented by variable *busElement*). All three variables (*element*, *busElement*, and *gElement*) refer to the same element object. A spatial reference object is derived from the polyline object.

When you test the draw bus route functionality, some routes are not in the extent of the Flagstaff parcel layer. You need to zoom out to see them.

3.5 Clearing elements from the map

Clearing elements from the map is quite straightforward. The DeleteAllElements() method on the IGraphicsContainer interface can be called to carry out the task. After calling this method, you must refresh the map to see the result. The following code is self-explanatory:



```
Private Sub btnClearMap_Click(...) Handles btnClearMap.Click
    Dim gc As IGraphicsContainer = My.ArcMap.Document.FocusMap
    gc.DeleteAllElements()

    My.ArcMap.Document.ActivatedView.Refresh()
End Sub
```

You may also create an add-in button in the CityGIS toolbar to clear the map.

Summary and Highlights

Graphic elements may be used to temporally represent features on a map, but they are not features stored in feature classes. Elements can be saved within map documents (.mxd files). To instantiate an element object, you must choose an appropriate element co-class. After instantiating an element, you must set a geometric object to it. The Geometry property is on the IElement interface. If a geometric object has a different spatial reference from the current map's spatial reference, a spatial reference object must be created and set to the SpatialReference property residing on the IGraphicElement interface of the element. You need to create a proper symbol object and set it to an element object to render the element. The default interface of each element co-class has a Symbol property.

To display an element in a map (data frame), you need to add the element into the map. The AddElement() method is on the IGraphicsContainer interface of the map object. After adding an element into the map, you can call the PartialRefresh() method to quickly refresh the map display.

A rubber band object allows you to track a desired geometric object on a map. The RubberBand abstract class has many co-classes each for a geometric type. The TrackNew() method on the IRubberBand interface must be called to track a desired geometric shape.

You can instantiate a polyline object from the Polyline co-class. To add points into a polyline, you have to use the IPointCollection interface since the AddPoint() method is no this interface.

You can create dockable windows in an ArcGIS desktop add-in project. A dockable windows object is managed by the application object. You can call the GetDockableWindow() method on the IDockableWindowManager of the application object to obtain a dockable window object. Since the GetDockableWindow() object requires a UID object as an argument, you must create a UID object in order to get a dockable window. You can instantiate a UID object from the UID co-class in the esriSystem assembly, and then set its Value property.

Exercises

1. Answer the following questions.
 - 1) What assembly is the Element class included?
 - 2) What assembly is the Geometry class included?
 - 3) Can you name three or more co-classes of the Element abstract class?
 - 4) Can you name three or more co-classes of the Geometry abstract class?
 - 5) Can you name three or more co-classes of the Symbol abstract class?
 - 6) If you have a line element object pointing to ILineElement interface, how do you set a polyline object to its Geometry property?
 - 7) If you have a marker element object pointing to the IMarkerElement interface, how do you set a point object to its Geometry property?
 - 8) Which interface of an element object provides the Symbol property?
 - 9) In order to create a marker element, what other objects are needed?
 - 10) In order to create a line element, what other objects are needed?
 - 11) How do you create a color object?
 - 12) How do you create a simple marker symbol object?
 - 13) How do you create a simple line symbol object?
 - 14) How do you create a simple fill symbol object?
 - 15) Can you name three interfaces of a map object that you have used?
 - 16) How do you get a point object from a mouse clicking location?
 - 17) Can you name three or more co-classes of the RubberBand abstract class?
 - 18) How do you get a polyline object from a rubber line object?
 - 19) In order to create a polyline from a rubber line object, what other objects are needed?
 - 20) How do you create rubber polygons?
 - 21) Why do we use the PartialRefresh method instead of the Fresh() method of the map object after adding an element to a map?
 - 22) How do you launch a dockable window using an add-in button?
 - 23) After creating a new polyline object that points to the IPolyline interface, how do you add points into it?
 - 24) Since you can create a new polyline object and add points to it, can you create a new polygons and add points to it? How would you do it?
 - 25) How do you create a stream reader object and used it to open a text file?
 - 26) How do you read a line from a stream reader object?
 - 27) How do you split a comma delimited line of string into data values?
 - 28) How does the ReadRouteCoordinates() procedure return two results to its client?
2. Create an ArcMap add-in tool in your CityGIS project. Name the tool "AreaMeasuringTool" with caption "Measure Area" and tooltip text "Area measuring tool". Use image:  (in folder "Images" on the CD-ROM) for the tool. This tool will be used to measure areas by tracking polygon features such as parcels and public open spaces. When a polygon is tracked, a polygon element is added into the map and a message box pops up to report the perimeter (in feet) and area (in acres). Please render the polygon element with a red border (rgb: 255, 0, 0) and a light green (rgb values: 128, 255, 168) fill color.

Tips:

- 1) Refer to the simplified model diagrams that you used while creating the bus route tool.
- 2) Use the RubberPolygon co-class to instantiate a rubber band object, and use this object to track a polygon shape in the tool's mouse down event procedure.

- 3) You may need to create a function named `CreateFillSymbol()` that takes a fill color object (IColor interface), a border color object (IColor interface), and a border width (Double) as input. The function must return a simple fill symbol object with ISimpleFillSymbol interface.
- 4) You can need to reuse the `CreateColor()` function created previously.
- 5) You will need to instantiate a polygon element object from the PolygonElement coclass and set its Geometry and Symbol properties. The Symbol property is on the IFillShapeElement interface (of FillShapeElement abstract class).
- 6) Polygon area is on the IArea interface, and polygon perimeter is on ICurve interface in the Curve abstract class. ICurve interface is inherited by IPolyCurve interface which is further inherited by IPolygon interface. Refer to the GeometryObjectModel.pdf (page 1) to figure out how to get polygon area and perimeter.
- 7) To test the tool, you must load a projected map layer. Please use the "Parcel" feature class from the personal geodatabase Flagstaff.mdb on the CD-ROM. This feature class is projected to SPCS AZ central zone on NAD 1983 with map unit foot. An acre is 43560 square feet.